Lab 3 Instructions

PUBH 523/623

Nicky Wakim

Directions

Please turn in your `.qmd` and `.html` file on Brightspace. Please use the same naming convention as practice assignments: `lastname_firstinit_lab_03.qmd` and `lastname_firstinit_lab_03.html`.

You can download the `.qmd` file for this lab here.

The above link will take you to your **editing** file. **Please do not remove anything from this editing file!!** You will only add your code and work to this file.

Purpose

The purpose of this lab is to build your composite score(s) and summarize your data. In Lab 2, you filtered to your population, selected and renamed your variables, and handled missing data. Now you'll clean up one last data issue (extra white space), build your composite score(s), and look at what your variables tell us: quick numeric summaries, a polished summary table, and summaries grouped by a variable of interest. **This is meant to build your comfort translating a numeric result back into a plain-language sentence someone outside this class could understand.**

This lab uses Lessons 16-29 (especially data transformations, missing data, and all four data summarization lessons). The only new function is `trimws()`, which is explained when you need it.

Before you start

How to use these instructions:

- Each section ends with a red **Task** box. Each task box is copied into your editing file, and your code and writing go right below it.

- Inside each section, the **numbered steps** walk you through the task in order. Do them one at a time and run your code after each step.
- **Use the exact object and variable names given in each task** (like `wbns_05_composite` and `ease_free_food_cat`). Lab 4 picks up from these names.
- When the instructions are different for each project, you will see **tabs**. Click on your project's tab. Once you pick a tab, every set of tabs on this page will switch to your project.
- Keep the WBNS questionnaire open while you work. You'll need it for the composite score.

Functions you'll use in this lab

Each function name links to the lesson slide where it's taught, in case you want a refresher. `trimws()` is new, so it links to R's help page instead.

Function	What it does	Lesson
`import()`	Read a data file into R	Lesson 17
`here()`	Build a file path that starts at your R project folder	Lesson 12
`dim()`	Print the number of rows and columns	Lesson 16
`levels()`	Show a factor's categories, in quotes	Lesson 23
`trimws()`	Remove extra white space (new in this lab)	R documentation
`mutate()`	Create or change variables	Lesson 22
`case_when()`	Assign values based on conditions	Lesson 23
`tabyl()`	Count each category, or cross-tabulate two variables	Lesson 28
`summary()`	Quick numeric summary of a variable	Lesson 27
`replace_na()`	Fill in NAs with a value (Project 3, WIC)	Lesson 25
`drop_na()`	Remove rows with NA in the columns you list	Lesson 25
`select()`	Keep only certain columns	Lesson 22
`skim()`	Quick summary of every variable	Lesson 27
`fct_relevel()`	Put a factor's categories in order	Lesson 23
`tbl_summary()`	Build a polished summary table	Lesson 28
`group_by()`	Tell the next function to work within groups	Lesson 29
`get_summary_stats()`	Summary statistics, overall or by group	Lesson 29

`export()`	Save data to a file	Lesson 17

Lab activities

Restate your selected project

Here is the table from Lab 2, with three new rows at the bottom: the names your main variables have **after** you renamed them in Lab 2, and the names of the composite score(s) and categories you'll create in this lab.

	Project 1	Project 2	Project 3
Project Question	For individuals under the federal poverty line, how do food support systems help individuals' food security?	For individuals who identify as LGBTQ, how does trust and/or fair treatment in healthcare affect self-reported health?	For individuals with school-aged children and below the federal poverty line, how do benefits received affect their mental health?
Population	People under 100% federal poverty line (FPL) and received free groceries or meals in the past 12 months	People who identify as LGBTQ and have a healthcare provider. (You can change this population if you want. For example, you may want to focus on how trust and fair treatment may improve health for people with larger bodies. Check out Q70F for options!)	People who have at least one child (aged 5-18 years old) in the household and are under the 100% FPL
Population variables	`FPL_LT50`, `FPL50_100`, `Q25`, `Q25A`, `Q26`, `Q26A`	`LGBTQ`, `Q108`	`ANY_CHILDHH_5_18`, `FPL_LT50`, `FPL50_100`
Main variables	`FOOD_INSEC`, `Q26D_1`, `Q26D_2`, `Q26D_3`, `Q26D_4`	`Q108B_2`, `Q108B_1`, `Q27`	`Q30_1`, `Q30_2`, `Q30_3`, `Q30_4`, `Q30_5`, `Q30_6`, `Q53_A`, `Q53_B`, `Q53_C`, `Q53_D`, `Q53_E`, `Q53_G`, `Q53_H`, `Q53_I`, `Q53_J`, `Q11`, `Q31_4`

	Project 1	Project 2	Project 3
Additional variables	`Q23`, `PPETHM`, `Q37`, `NUM_CHILD_HH`	`Q70F_4`, `Q70F_5`, `PPETHM`, `Q28O`	`NUM_CHILD_HH`, `Q1`, `Q37`, `PPETHM`, `Q27`
Composite score	Composite score of ease of access to free groceries or meals (combined score from `Q26D_`)	Level of trust, courtesy, and respect around treatment (combined score from `Q108B_1` and `Q108B_2`)	Total mental health (combined scores from `Q30_`), Total benefits (combined scores from `Q53_`, `Q11`, and `Q31_4`)
Main variable names after Lab 2	`food_insecure`, `ease_find_place`, `ease_transport`, `ease_variety`, `ease_timely`	`trust_provider`, `courtesy_respect`, `self_rated_health`	`mh_nervous`, `mh_hopeless`, `mh_restless`, `mh_sad`, `mh_effort`, `mh_worthless`, the seven `benefit_` variables, `medicaid_chip_q53`, `medicaid_chip_q31`, `housing_q53`, `housing_q11`
Composite score name(s)	`ease_free_food`	`trust_courtesy_respect`	`mental_health_score` and `total_benefits`
Categorical version name(s)	`ease_free_food_cat`	`trust_courtesy_respect_cat`	`mental_health_score_cat` and `total_benefits_cat`

Here is how the objects in this lab build on each other:

Section	Object you create	What it holds
Import your saved data from Lab 2	`wbns_04_no_na`	Your clean dataset from Lab 2
Clean up extra white space	`wbns_04_trimmed`	Same data, with extra spaces removed from the categories
Construct the composite score	`wbns_05_composite`	Your new composite score(s)
Quick summaries	`quick_summaries`	A `skim()` summary of your main variables
Create categories	`wbns_06_cat`	A categorical version of each composite score
Build a summary table	`summary_table_overall`	A polished `tbl_summary()` table

Section	Object you create	What it holds
Grouped summaries	`summary_grouped`	Your composite score summarized within each group

! Task

In your editing file, fill in the short list under "Restate your selected project" for your project: your project number, project question, population, main variable names (after Lab 2), and the composite score name(s) you'll create. This will help you (and me hehe) keep track of your project as you work through this lab.

Import your saved data from Lab 2

Your Lab 2 `.rda` file holds several datasets, so we need to tell `import()` which one we want.

1. Find your Lab 2 `.rda` file in your `data_project/data_processed` folder and note its exact name.
2. Use `import()` with `here()`, and add the `which` argument with the name of the dataset in quotes. The `which` argument is new: it tells `import()` which dataset to pull out of a file that holds several. Replace the file name with yours:

```
wbns_04_no_na <- import(
  here("data_project", "data_processed", "wbns_data_lab_02_work.rda"),
  which = "wbns_04_no_na"
)
```

3. Use `dim()` to print the number of rows and columns. The number of rows should match what you got at the end of Lab 2.

i Projects 1 and 3: you will still see missing data, and that's okay!

Some variables in `wbns_04_no_na` are still very missing on purpose. They are missing because of skip logic (people who were never asked the question because an earlier answer already told us), and we didn't drop them in Lab 2.

- **Project 1:** `Q25A` and `Q26A` are only asked of people who said no to `Q25` or `Q26`. You don't need them for your composite score. Leave them as they are, and don't drop rows because of them.
- **Project 3:** `medicaid_chip_q53`, `housing_q53`, `housing_q11`, and `benefit_wic`. You'll handle these when you build `total_benefits`.

You'll see this missing data again in your quick summaries. Note it, but don't remove it.

> **!** Task
>
> From the `.rda` file you exported at the end of Lab 2, import `wbns_04_no_na` into R using `import()` from the `rio` package. Save it as an object called `wbns_04_no_na`. Use `dim()` to print the number of rows and columns to verify the import.

Clean up extra white space

White space is any character you can't see: spaces, tabs, and line breaks. Sometimes survey data comes with an extra space at the end of a category, so the answer is stored as `"Yes "` instead of `"Yes"`. You can't see the difference when R prints the data, but **R treats them as two different values**. That means `benefit_snap == "Yes"` would be `FALSE` for every `"Yes "`, and your composite score would quietly turn into `NA`. (This is the same idea as the "spelling/spacing is important" slides in Lesson 23.)

Some of the WBNS variables have extra spaces like this, especially in Project 3. We'll remove them from **every** factor before building the composite score. It's a good habit even if your project doesn't have the problem.

The function that removes extra white space is `trimws()` (**trim** **w**hite **s**pace). It removes spaces from the beginning and end of text, but leaves the spaces between words alone:

```
trimws("Yes ")
```

```
[1] "Yes"
```

```
trimws("  Don't know  ")
```

```
[1] "Don't know"
```

```
trimws("Some of the time")
```

```
[1] "Some of the time"
```

1. **Look for extra spaces.** Use `levels()` (Lesson 23) on one of your main variables to see its categories in quotes. In quotes, a trailing space is visible: `"Yes "` instead of `"Yes"`. Use the variable from your project's tab below.
2. **Remove extra spaces from every factor.** This code uses the same pattern as the code you used in Lab 1 to remove the numbers in parentheses from the categories. Instead of `gsub()`, it uses `trimws()`. Copy it into your editing file as is:

```
wbns_04_trimmed <- wbns_04_no_na |>
  mutate(
    across(
      where(is.factor), ~ {
        levels(.) <- trimws(levels(.))
        .
      }
    )
  )
```

`across(where(is.factor), ...)` means "do this to every factor variable," and `levels(.) <- trimws(levels(.))` replaces each factor's categories with their trimmed versions. If a variable had both `"Yes"` and `"Yes "` as categories, they are now combined into one `"Yes"`.

3. **Check your work.** Run `levels()` on the same variable again, using `wbns_04_trimmed`. The extra spaces should be gone. Then use `dim()` to confirm the number of rows and columns didn't change.

Project 1

Check `levels(wbns_04_no_na$ease_find_place)` before and `levels(wbns_04_trimmed$ease_find_place)` after.

Project 2

Check `levels(wbns_04_no_na$trust_provider)` before and `levels(wbns_04_trimmed$trust_provider)` after.

Project 3

Check `levels(wbns_04_no_na$mh_nervous)` before and `levels(wbns_04_trimmed$mh_nervous)` after. The mental health items are where the extra spaces are: look closely at the quotes in the "before" output, and you'll see categories like `"Most of the time "`. Without trimming, `mh_nervous == "Most of the time"` would never be `TRUE`.

> Did Lab 2 miss anything because of extra spaces?
>
> If a category like `"Don't know "` had a trailing space, then `replace_with_na()` in Lab 2 may not have matched it. That's okay: those values are still categories (not `NA`) in your data. When you build your composite score, they won't match any line in your `case_when()`, so they'll become `NA` there, and the `NA` checks in the next section will catch

them.

! Task

Use `trimws()` to remove extra white space from the categories of every factor variable. Save the result as `wbns_04_trimmed`. Use `levels()` on one of your main variables before and after to show the extra spaces are gone, and use `dim()` to confirm no rows or columns were lost.

Construct the composite score

Caution

Please make sure you are referencing the PDF with the questionnaire when creating your composite score. There are important descriptions and caveats in the questionnaire!

Your project must create a brand-new variable that combines information from at least two source columns using `mutate()`. Start from `wbns_04_trimmed`. The table at the top of this lab lists the variables to combine, and your project's tab below walks you through it.

The general recipe (the same for every project):

1. **Confirm the response scale** for each item in the questionnaire. Is it Yes/No? A 5-point scale? What order are the levels in?
2. **Convert each item to a number** using `case_when()` inside `mutate()` (Lesson 23). Write one line for each category: `item == "Category" ~ number`. Save each converted item as a **new** variable ending in `_num` (e.g., `ease_find_place_num`), so you can compare it to the original.
    - Any value that doesn't match one of your conditions (including `NA`) automatically becomes `NA`. That's what we want!
3. **Check each conversion** with `tabyl()`, using the pattern from Lesson 25: `wbns_05_composite |> tabyl(ease_find_place, ease_find_place_num)`. Every category should line up with the number you intended.
4. **Add the converted items together** into one new composite variable inside `mutate()`, using the required name from your project's tab.
5. **Sanity-check the result:** use `summary()` on your composite score, and confirm the minimum and maximum are within the possible range. Does a higher score mean *more* of what you think it means?
    - **Check for `NA`s.** You removed missing values in Lab 2, so `ease_free_food`, `trust_courtesy_respect`, and `mental_health_score` should have **no** `NA`s. If

`summary()` shows an `NA's` count, some category didn't match any line in your `case_when()`. Usually that's a "Refused" or "Don't know" that wasn't turned into `NA` in Lab 2 Part A, or a spelling mismatch. Use `tabyl()` on the original item and its `_num` version to find it. (Project 3's `total_benefits` can have a few `NA`s; step 5 in your tab handles those.)

> ⚠ Why not `as.numeric()`?
>
> You might see code online that converts survey answers with `as.numeric()`. **Don't use it for this lab.** Our variables are factors, and `as.numeric()` gives each level's *position* in the list of levels, **not** the survey's number for that answer. If a factor's first level is `"Refused"`, then `"Very difficult"` becomes 2 instead of 1, and your whole score shifts without any error message. `case_when()` lets you say exactly which number goes with which answer.

To keep it easy to debug, build your composite in **two steps**:

- **Step A:** one `mutate()` that creates all of your `_num` variables (your tab has the code to start from). Check them with `tabyl()` before moving on.
- **Step B:** a second `mutate()` that adds them together. You can add on to the same object, like this (Project 1 shown):

```
wbns_05_composite <- wbns_05_composite |>
  mutate(
    ease_free_food = ease_find_place_num + ease_transport_num +
      ease_variety_num + ease_timely_num
  )
```

  Starting a pipeline with the same object you're saving to (`wbns_05_composite <- wbns_05_composite |> ...`) updates that object with your new variable.

Project 1

Composite name: `ease_free_food`

Your composite combines the four `ease_` items, which each ask how difficult or easy a different aspect of accessing free groceries/meals was (finding a place, finding transportation, variety of food, and getting there in a reasonable time). Per the questionnaire, the response scale is **Very difficult (1), Difficult (2), Neither difficult nor easy (3), Easy (4), Very easy (5), Don't know (6)**.

1. You already turned `"Don't know"` into `NA` in Lab 2, so you only need to convert five categories.

2. Convert each item with `case_when()`. Here is the first one; write the other three the same way:

```
wbns_05_composite <- wbns_04_trimmed |>
  mutate(
    ease_find_place_num = case_when(
      ease_find_place == "Very difficult" ~ 1,
      ease_find_place == "Difficult" ~ 2,
      ease_find_place == "Neither difficult nor easy" ~ 3,
      ease_find_place == "Easy" ~ 4,
      ease_find_place == "Very easy" ~ 5
    )
    # add ease_transport_num, ease_variety_num, and ease_timely_num here
  )
```

   Check that the category spelling matches your `tabyl()` output exactly.

3. Higher values already mean *easier*, so no reverse-coding is needed.

4. Add the four `_num` items together into `ease_free_food`.

5. With 4 items on a 1-5 scale, the possible range is 4 (very difficult across the board) to 20 (very easy across the board). Check that your `summary()` falls in this range.

Project 2

Composite name: `trust_courtesy_respect`

Your composite combines `courtesy_respect` (Q108B_1: "I am treated with courtesy and respect by my personal health care provider") and `trust_provider` (Q108B_2: "I trust my personal health care provider"). Per the questionnaire, both share the same response scale: **Never true (1), Rarely true (2), Sometimes true (3), Often true (4), Always true (5)**.

1. Convert each item with `case_when()`. Here is the first one; write the second the same way:

```
wbns_05_composite <- wbns_04_trimmed |>
  mutate(
    courtesy_respect_num = case_when(
      courtesy_respect == "Never true" ~ 1,
      courtesy_respect == "Rarely true" ~ 2,
      courtesy_respect == "Sometimes true" ~ 3,
      courtesy_respect == "Often true" ~ 4,
      courtesy_respect == "Always true" ~ 5
    )
```

```
    # add trust_provider_num here
  )
```

Check that the category spelling matches your `tabyl()` output exactly.

2. Higher values already mean *more* trust/courtesy/respect, so no reverse-coding is needed.
3. Add the two `_num` items together into `trust_courtesy_respect`. It will range from 2 (never true on both) to 10 (always true on both). Check that your `summary()` falls in this range.
4. *Optional:* if you kept `Q70F_4` and `Q70F_5`, you could build a second, related composite capturing whether the respondent felt judged unfairly based on gender identity or sexual orientation, and compare it against your trust/courtesy/respect score.

Project 3

Composite names: `mental_health_score` and `total_benefits`

This project asks for *two* composite scores, and both have a wrinkle worth slowing down for. Don't try to do it all in one `mutate()`. Build it up piece by piece.

Mental health composite (`mental_health_score`, from the six `mh_` items):

These six items ask how often in the past 30 days the respondent felt nervous, hopeless, restless/fidgety, so sad that nothing could cheer them up, that everything was an effort, or worthless. Per the questionnaire, the scale is **All of the time (1), Most of the time (2), Some of the time (3), A little of the time (4), None of the time (5)**. That means a *lower* number = more frequent distress (worse mental health), and a *higher* number = less frequent distress (better mental health).

1. Decide which direction you want your composite to run, and say so in your write-up:
   - **Option A:** Use the survey's numbers as-is (`"All of the time" ~ 1`, …, `"None of the time" ~ 5`). Your total will run low = worse, high = better.
   - **Option B:** Flip the numbers in your `case_when()` (`"All of the time" ~ 5`, …, `"None of the time" ~ 1`). Your total will run low = better, high = worse, which many people find more intuitive.
2. Convert all six items with `case_when()`, using the same direction for every item. Here is the first one, using Option A:

```
wbns_05_composite <- wbns_04_trimmed |>
  mutate(
    mh_nervous_num = case_when(
      mh_nervous == "All of the time" ~ 1,
      mh_nervous == "Most of the time" ~ 2,
```

```
      mh_nervous == "Some of the time" ~ 3,
      mh_nervous == "A little of the time" ~ 4,
      mh_nervous == "None of the time" ~ 5
    )
    # add the other five mh_ items here
  )
```

3. Add the six `_num` items together into `mental_health_score`. The possible range is 6 to 30.

Total benefits composite (`total_benefits`) — this is the tricky part:

The `Q53_` series asks about several *different* benefit programs, each a separate Yes/No question, not a scale. Two of them have **built-in skip logic** (which you saw in your `gg_miss_fct()` plots in Lab 2):

- `medicaid_chip_q53` (Q53_B) is *only asked* of people who did **not** already report Medicaid/CHIP coverage in Q31 (item d). If someone said `"Covered"` on `medicaid_chip_q31`, they were skipped past Q53_B, so it is missing for them, even though they clearly do have this benefit. People who said "Not Covered" or "Not Sure" on Q31 **were** asked Q53_B.
- `housing_q53` (Q53_C) has the same issue with `housing_q11` ("is your household paying lower rent because a government program is paying part of the cost?"). Anyone who said `"Yes"` on Q11 was skipped past Q53_C. Everyone else was asked Q53_C, including homeowners who never saw Q11.

Here's how the two questions in each pair fit together:

What they said on Q31 item d (or Q11)	Asked Q53_B (or Q53_C)?	Receives the benefit?
"Covered" (or "Yes")	No, skipped	**Yes** (1)
Anything else, or not asked	Yes	Use their Q53 answer: "Yes" = 1, "No" = 0
Anything else, and Q53 is `NA` (e.g., "Don't know")	Yes	Unknown (`NA`)

1. **Combine each skipped pair into one 0/1 indicator.** Count the person as receiving the benefit (1) if *either* question says yes. Count them as not receiving it (0) only if they were asked Q53 and said `"No"`. Here is Medicaid/CHIP:

```
medicaid_chip = case_when(
  medicaid_chip_q53 == "Yes" | medicaid_chip_q31 == "Covered" ~ 1,
  medicaid_chip_q53 == "No" ~ 0
)
```

If both questions are missing, neither condition is true, so `medicaid_chip` becomes `NA`. Write `housing` the same way, using `housing_q53` and `housing_q11`.

2. **Convert the other benefit items** to 0/1 indicators with `case_when()` (`"Yes" ~ 1`, `"No" ~ 0`). Name each one with `_num` at the end (e.g., `benefit_snap_num`).

   - `benefit_school_meals` is only asked of households with a child aged 5-18. Your population filter already requires that, so everyone in your sample should have an answer.
   - `benefit_wic` is only asked of households with a child aged 0-5, which your population filter does **not** guarantee. WIC may be missing for many people because they weren't asked, not because they refused. Decide whether to include WIC in your total at all. If you do, use `replace_na()` from Lesson 25 to fill in missing `benefit_wic_num` values with 0 instead of dropping those rows, and explain your choice. (Because you turned "Don't know" into `NA` in Lab 2, this also counts the few people who answered "Don't know" about WIC as not receiving it. Mention that in your explanation.)

3. **Check** each indicator with `tabyl()`. For the combined pairs, check against both originals, e.g. `wbns_05_composite |> tabyl(medicaid_chip_q53, medicaid_chip_q31)` next to `wbns_05_composite |> tabyl(medicaid_chip)`.

4. **Add every 0/1 indicator together** into `total_benefits`, a count of how many *different* types of benefits the household receives.

5. **Remove rows with an unknown score.** If any indicator is `NA` (for example, someone who said "Don't know" on Q53_B and wasn't covered on Q31), `total_benefits` will be `NA`, because `1 + NA` is `NA`. The same goes for `mental_health_score` if one of the `mh_` items is `NA` (for example, a "Refused" that wasn't turned into `NA` in Lab 2). Use `dim()`, then remove those rows, then use `dim()` again:

```
dim(wbns_05_composite)
wbns_05_composite <- wbns_05_composite |>
  drop_na(total_benefits, mental_health_score)
dim(wbns_05_composite)
```

   This should remove only a small number of rows. If it removes a lot, check your `_num` variables with `tabyl()`: a category that didn't match any line in your `case_when()` (like an untrimmed `"Yes "`) is the usual cause.

! Task

Create your new composite score(s) using `case_when()` and `mutate()`, following the steps for your project. Save this as a new object called `wbns_05_composite`.

1. Use `tabyl()` to check at least two of your converted (`_num`) variables against the

originals.
2. Use `summary()` to display a summary of each composite score.
3. Briefly explain (1-2 sentences) whether the range and direction of your composite score make sense. (Project 3: say which direction you chose for `mental_health_score`, whether you included WIC, and how many rows you removed because `total_benefits` or `mental_health_score` was `NA`.)

Quick summaries of your variables

Before building polished tables, get a quick feel for your variables with `skim()` (Lesson 27). This is your chance to catch problems, like a composite score outside its possible range or a variable with unexpected missing data, before you summarize anything for real.

1. Start from `wbns_05_composite` and use `select()` to keep only the variables listed in your project's tab below.
2. Pipe the result into `skim()`, and save it as `quick_summaries`:

```
quick_summaries <- wbns_05_composite |>
  select(variable_1, variable_2) |>
  skim()
```

   Replace `variable_1, variable_2` with the variables in your tab.
3. Print `quick_summaries` by typing its name on its own line.
4. Read the output. `skim()` makes one section for **factor** variables and one for **numeric** variables. Look for:
   - **`n_missing`:** Your composite score(s) should have 0 (you removed missing values in Lab 2 and, for Project 3, with `drop_na(total_benefits, mental_health_score)`).
   - **`p0` and `p100`:** These are the minimum and maximum. They must be inside your composite score's possible range from the last section.
   - **`mean`, `p50` (median), and `hist`:** Where do most people's scores fall? You'll use this to choose categories in the next section.
   - **`top_counts`** (factors): Do the categories look right? Is any category much larger or smaller than you expected?
5. Write 2-3 sentences: Is everything within the expected range, and is anything surprising? Where does most of your composite score fall?

Project 1

Select `food_insecure`, `ease_find_place`, `ease_transport`, `ease_variety`, `ease_timely`, and `ease_free_food`.

`ease_free_food` should be between 4 and 20.

Project 2

Select `self_rated_health`, `courtesy_respect`, `trust_provider`, and `trust_courtesy_respect`.

`trust_courtesy_respect` should be between 2 and 10.

Project 3

Select `mental_health_score`, `total_benefits`, `medicaid_chip`, and `housing`.

`mental_health_score` should be between 6 and 30. `total_benefits` should be between 0 and 9 (or 0 and 8, if you didn't include WIC).

> **!** Task
>
> Use `select()` and `skim()` to generate a quick summary of your main variables and composite score(s) from `wbns_05_composite`. Save the result as `quick_summaries`, and print it. Write 2-3 sentences describing what you see: Is everything in the expected range? Is anything surprising? Where do most composite scores fall?

Create categories for your composite score(s)

A composite score is precise, but it's hard to explain at a glance: "the average score was 14.3" means less to a general audience than "most people reported easy access." In this section, you'll create a **categorical version** of each composite score, so you have both versions for Lab 4 and your final project.

We'll use `case_when()` again, this time with ranges of numbers instead of categories. This is the same idea as the income example in Lesson 23 (`income < 30000 ~ "<$30,000"`).

1. **Choose your cut points.** Start with the suggested categories in your project's tab. Use your quick summaries to check that each category will have a reasonable number of people in it. If one category would be nearly empty, it's okay to adjust the cut points or combine two categories. Just explain why.

2. **Create the categories** with `case_when()` inside `mutate()`. Write one line per category, using `>=` and `<=` with `&` to give each range. Use the **exact** new variable name from your tab (it ends in `_cat`). For example, for Project 1:

```
wbns_06_cat <- wbns_05_composite |>
  mutate(
    ease_free_food_cat = case_when(
      ease_free_food >= 4 & ease_free_food <= 9 ~ "Difficult access",
      ease_free_food >= 10 & ease_free_food <= 15 ~ "Moderate access",
      ease_free_food >= 16 & ease_free_food <= 20 ~ "Easy access"
    )
  )
```

   Make sure every possible score fits into exactly one range, with no gaps (for example, 9 and 10 are in different categories, and nothing falls between them).

3. **Put the categories in order.** `case_when()` gives you text, and R will sort text alphabetically ("Difficult", "Easy", "Moderate"). That would put your categories out of order in every table and plot. Use `fct_relevel()` (Lesson 23) to turn the new variable into a factor with the categories in a meaningful order, from lowest to highest:

```
wbns_06_cat <- wbns_06_cat |>
  mutate(
    ease_free_food_cat = fct_relevel(
      ease_free_food_cat,
      "Difficult access", "Moderate access", "Easy access"
    )
  )
```

   Spell each category exactly as you did in `case_when()`. If you get a warning about "unknown levels," a category name doesn't match.

4. **Check your categories.** Use `tabyl()` to cross-tabulate the score with its new categorical version (Lesson 28):

```
wbns_06_cat |> tabyl(ease_free_food, ease_free_food_cat)
```

   Each row (one score) should have counts in **only one** category column, and there should be **no** `NA_` column. An `NA_` column means some scores didn't fit into any of your ranges.

5. **Count people in each category** with `tabyl()` on just the new variable (e.g., `wbns_06_cat |> tabyl(ease_free_food_cat)`), and check that the categories print in the order you set.

Project 1

Create **one** categorical variable: `ease_free_food_cat` from `ease_free_food` (possible range 4-20).

Suggested categories, in order:

Scores	Category
4-9	"Difficult access"
10-15	"Moderate access"
16-20	"Easy access"

Project 2

Create **one** categorical variable: `trust_courtesy_respect_cat` from `trust_courtesy_respect` (possible range 2-10).

Suggested categories, in order:

Scores	Category
2-5	"Low trust"
6-8	"Moderate trust"
9-10	"High trust"

Project 3

Create **two** categorical variables. You can create both in the same `mutate()`, separated by a comma. Lab 4 uses both.

`total_benefits_cat` from `total_benefits` (possible range 0-9):

Number of benefits	Category
0-1	"0-1 benefits"
2-3	"2-3 benefits"
4-5	"4-5 benefits"
6-9	"6+ benefits"

If very few people have 6 or more benefits, combine the last two into "4+ benefits" (4-9).

`mental_health_score_cat` from `mental_health_score` (possible range 6-30). The suggested categories depend on the direction you chose when you built the score:

Option A (high = better)	Option B (high = worse)	Category
6-14	22-30	“Frequent distress”
15-22	14-21	“Some distress”
23-30	6-13	“Little distress”

Either way, order the levels as `"Frequent distress"`, `"Some distress"`, `"Little distress"`.

Another way to make categories: `cut()` (optional)

`cut()` is a function made specifically for turning a numeric variable into categories. It's not covered in our lessons, so `case_when()` is the main method in this lab, but you're welcome to use `cut()` instead. You'll also see it a lot in other people's code, so here's how it works.
You give `cut()` the **break points** between categories and a **label** for each category:

```
wbns_06_cat <- wbns_05_composite |>
  mutate(
    ease_free_food_cat = cut(
      ease_free_food,
      breaks = c(3, 9, 15, 20),
      labels = c("Difficult access", "Moderate access", "Easy access")
    )
  )
```

The tricky part is the break points. By default, each category **includes its upper break and excludes its lower break**. So `breaks = c(3, 9, 15, 20)` makes three ranges: more than 3 up to 9 (4-9), more than 9 up to 15 (10-15), and more than 15 up to 20 (16-20). That's why the first break is 3, not 4: a break of 4 would leave out a score of 4. This “off-by-one” mix-up is the most common mistake with `cut()`, so always check your categories with `tabyl()` the same way as step 4.
Here are the break points that match the suggested categories:

Project	Score	`breaks`
1	`ease_free_food`	`c(3, 9, 15, 20)`
2	`trust_courtesy_respect`	`c(1, 5, 8, 10)`
3	`mental_health_score` (Option A)	`c(5, 14, 22, 30)`
3	`mental_health_score` (Option B)	`c(5, 13, 21, 30)`
3	`total_benefits`	`c(-1, 1, 3, 5, 9)`

For `total_benefits`, the first break is -1 so that 0 is included.
One nice thing about `cut()`: it returns a factor with the categories already in the order you listed them in `labels`, so you can usually skip step 3 (`fct_relevel()`). The `labels` always go from the lowest scores to the highest. The one exception is Project 3's mental health score with **Option B**: its labels are `c("Little distress", "Some distress", "Frequent distress")`, so you still need `fct_relevel()` to put "Frequent distress" first, like the other projects' order.

! Task

Use `mutate()` with `case_when()` (or `cut()`) to create the categorical version(s) of your composite score(s), using the required name(s) from your project's tab. Put the categories in a meaningful order with `fct_relevel()` (or, if you used `cut()`, with the order of your `labels`). Save the result as `wbns_06_cat`.

1. Use `tabyl()` to cross-tabulate each composite score with its categorical version, and confirm every score falls into exactly one category with no `NA`s.
2. Use `tabyl()` to show how many respondents fall into each category.
3. In 1-2 sentences, say which cut points you used, and why if you changed them from the suggestions.

Build a summary table

Now put your summaries into a table that would make sense to someone outside this class. This is the kind of table you might include in your final project. We'll use `tbl_summary()` from Lesson 28.

1. Start from `wbns_06_cat`, and use `select()` to keep only the variables in your project's tab below. `tbl_summary()` summarizes **every** column you give it, so this step controls what's in your table.
2. Pipe into `tbl_summary()`. Use the `label` argument to give **every** variable a readable name. When you're labeling more than one variable, put the labels inside `list()`, separated by commas:

```
summary_table_overall <- wbns_06_cat |>
  select(food_insecure, ease_free_food, ease_free_food_cat) |>
  tbl_summary(
    label = list(
      food_insecure ~ "Food insecure",
      ease_free_food ~ "Ease of access to free food (score, 4-20)",
      ease_free_food_cat ~ "Ease of access to free food"
```

```
    )
  )
```

3. Print `summary_table_overall` to see your table. Check that the categories are in the order you set, and that the labels would make sense to someone who has never seen your data.
4. *Optional:* use the `statistic` argument to show the mean (SD) for your numeric composite score instead of the default median (IQR), e.g., `statistic = ease_free_food ~ "{mean} ({sd})"`.

Project 1

Select `food_insecure`, `ease_free_food`, and `ease_free_food_cat`. (The example above is yours. Feel free to change the labels!)

Project 2

Select `self_rated_health`, `trust_courtesy_respect`, and `trust_courtesy_respect_cat`.

Project 3

Select `mental_health_score`, `mental_health_score_cat`, `total_benefits`, and `total_benefits_cat`.

In the label for `mental_health_score`, say which direction it runs (e.g., "Mental health score (6-30, higher = better)"), since a reader can't know that otherwise.

> **!** Task
>
> Use `select()` and `tbl_summary()` to build an overall summary table of your composite score(s) and their categories (plus `food_insecure` for Project 1 or `self_rated_health` for Project 2). Use the `label` argument to give every variable a readable name. Save this as `summary_table_overall`, and display it.

Grouped summaries

Your project question is about a relationship: how one thing relates to another. An overall summary can't show that. To see a relationship, compare your composite score **across groups** of another variable.

1. Start from `wbns_06_cat`. Use `group_by()` with your project's grouping variable (in your tab below).

2. Pipe into `get_summary_stats()` with your composite score and `type = "common"` (Lesson 29). Save the result as `summary_grouped`:

```
summary_grouped <- wbns_06_cat |>
  group_by(food_insecure) |>
  get_summary_stats(ease_free_food, type = "common")
```

3. Print `summary_grouped`. You'll get one row per group.
4. Read the output. Compare the `mean` and `median` across groups: do they differ, and by how much? Also look at `n`: a group with only a few people gives a less reliable summary.
5. Write 2-3 sentences interpreting what you see and connecting it to your project question. You can use this sentence pattern to get started: "Among [group 1], the median [score] was ***, compared to*** among [group 2]. This suggests that…" Remember what a higher score means for your composite!

Project 1

Group by `food_insecure`, and summarize `ease_free_food`. (The example above is yours.)

Does ease of access to free food look different for people who are food insecure and people who aren't?

Project 2

Group by `self_rated_health`, and summarize `trust_courtesy_respect`.

Does trust, courtesy, and respect look different across levels of self-rated health?

Project 3

Group by `total_benefits_cat`, and summarize `mental_health_score`.

Does mental health look different for households that receive more types of benefits? Remember which direction your mental health score runs (Option A or B) when you interpret it.

> **Note**
>
> *Optional:* You can also add `by =` to `tbl_summary()` to get a side-by-side table for each group (Lesson 29), e.g., `tbl_summary(by = food_insecure)` after selecting `food_insecure` and `ease_free_food`.

! Task

Use `group_by()` and `get_summary_stats()` to calculate summary statistics of your composite score for each level of your project's grouping variable. Save this as `summary_grouped`, and print it. Write 2-3 sentences interpreting what you see. Does the composite score differ across groups? Does this match what you'd expect given your project question?

Export your data and table

Save your work so you can use it in Lab 4. Just like in Lab 2, we'll save both objects into **one** `.rda` file.

1. Give `export()` a `list()` of the objects to save, using the pattern `name = object`.
2. Use `here()` to save the file in your `data_project/data_processed` folder:

```
export(
  list(
    wbns_06_cat = wbns_06_cat,
    summary_table_overall = summary_table_overall
  ),
  here("data_project", "data_processed", "wbns_data_lab_03_work.rda")
)
```

3. Check your `data_processed` folder to make sure the file is there.

! Task

Save `wbns_06_cat` and `summary_table_overall` in a `.rda` file in your `data_project/data_processed` folder. Name the file something that will help you identify it later, like `wbns_data_lab_03_work.rda`.

AI Use Statement

Using AI is not required in this class. If you *did* use an AI tool (ChatGPT, Claude, Copilot, Gemini, an RStudio assistant, etc.) on this lab, add a brief AI Use Statement. Include:

- **Which tool(s)** you used
- **What for** (a sentence or two, with section names)
- **Where** it contributed to any written answers (if at all)
- *You do NOT need to link your conversation.*

For example:

> *AI Use Statement: Claude was used to explain an error message in the "Create categories" section. All reasoning and answers are my own.*

OR

> *AI Use Statement: AI was not used in this assignment.*

Before using AI, check the table of allowed uses in the Generative AI Policy. AI can help you explain errors, debug, and style your code, but it should not write your analysis for you.

! Task

Write your AI Use Statement in your editing file.